

Feature Tracking and Dynamic Object Detection

A friendly walk-through of the method and the code

Mahdi Gheysari

Filippo Businaro

Riccardo Pesce

April 2026

Abstract

This is the long version of our report, written in plain English. If the two-page report is the *what* and *why*, this one is the *how*. We explain every step of the pipeline, show the actual code, and point out who wrote which file. The math and the parameter choices are all still here — we just stopped pretending that our code is a research paper.

Contents

1	What are we trying to do?	1
2	The idea, in plain English	2
3	The big picture	2
4	The method, step by step	3
4.1	Step 1 — Finding interesting points (SIFT)	3
4.2	Step 2 — Matching: finding the same point later	3
4.3	Step 3 — Getting rid of the camera’s drift	4
4.4	Step 4 — Deciding who’s on the object	4
4.5	Step 5 — Grouping the movers and drawing a box	5
5	A tour of the code	6
5.1	Who wrote what	6
5.2	The shared struct	6
5.3	<code>feature_tracker.cpp</code> (Mahdi)	7
5.4	<code>motion_segmenter.cpp</code> (Filippo)	7
5.5	<code>bbox_estimator.cpp</code> (Riccardo)	8
5.6	The two small files	9
6	Numbers	9
7	Where it breaks and why	10
8	How to run it	11
9	Who did what	11

1 What are we trying to do?

You get a short video (a folder of numbered frames). Somewhere in that video, *one thing* is moving — a bird, a car, a frog, a sheep, or a squirrel. Everything else is background. Our job is to look at the **first frame** and draw a box around that moving thing.

We cannot use deep learning. Only classical computer vision: corner detectors, feature descriptors, matching, clustering, geometry. The dataset gives us five categories, one ground-truth box per

category. We are graded on *Intersection-over-Union* (IoU) between our box and the ground truth. $\text{IoU} \geq 0.5$ counts as a true positive.

What we noticed about the data. Before writing any code, we just *looked* at the videos:

- In **bird**, **frog**, **sheep**, and **squirrel**, the camera barely moves. It is sitting on a tripod. Only the animal moves.
- In **car**, the camera does a slow left-to-right pan. The whole scene drifts a little in the same direction.

This is important because it tells us we don't need a fancy model of camera motion. We just need to remove a small, constant drift.

2 The idea, in plain English

Here is the whole trick in one sentence:

If you pick some interesting points on the first frame and try to find them again in every later frame, the background points will all move together (or not at all), and the object's points will stand out by moving differently.

So the plan is:

1. Put dots on interesting corners of every frame.
2. For each dot on frame 0, find the same dot in every later frame.
3. Figure out the “average” movement — that's the camera.
4. Subtract the camera movement. Whatever is left is real object motion.
5. Dots with a lot of leftover motion are on the moving object. Group them. Draw a box around them.

That's the entire method. The rest of this document is just explaining what “interesting corner”, “find the same dot”, “group them”, etc. actually mean in code.

3 The big picture

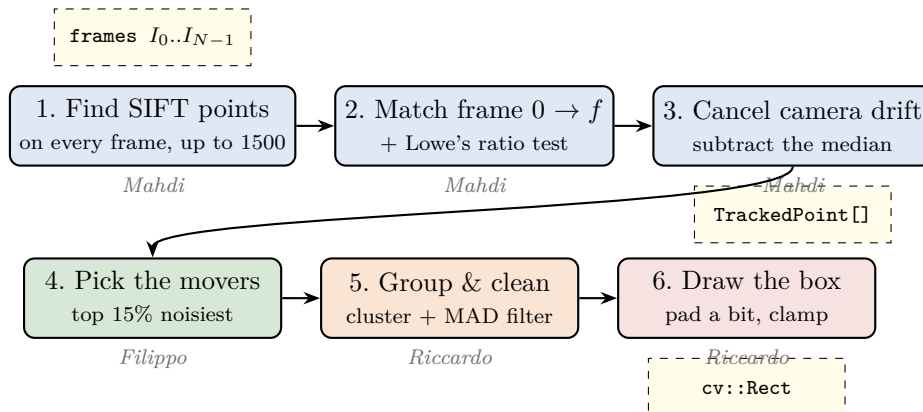


Figure 1: The whole pipeline. Blue = finding and matching points (`feature_tracker.cpp`). Green = deciding which points are on the object (`motion_segmenter.cpp`). Orange/red = grouping and boxing (`bbox_estimator.cpp`). The name under each stage is whoever wrote it.

4 The method, step by step

4.1 Step 1 — Finding interesting points (SIFT)

We need “interesting” pixels — ones we can find again later. A flat patch of sky is useless; a brick corner is great.

We use **SIFT**¹: `cv::SIFT::create(1500)` followed by `detectAndCompute`. For every frame this gives us up to 1500 *keypoints* (where the interesting spots are) and a 128-number *descriptor* for each one (basically a summary of what the neighbourhood around the keypoint looks like).

Why SIFT? Three reasons:

- **Scale-invariant.** SIFT looks at the image at several zoom levels and picks points that are “spotty” at some zoom. Zooming in or out doesn’t change what it picks.
- **Rotation-invariant.** Each descriptor is rotated so its dominant gradient points “up” before we measure it. So a rotated feature still looks the same.
- **Illumination-tolerant.** The descriptor is normalised, clipped at 0.2, and normalised again. That kills the effect of overall brightness or contrast changes — which matters when leaves cast moving shadows.

The PDF literally recommends “sparse local features + robust matching”. SIFT is the textbook answer,² and OpenCV’s own feature-matching tutorial uses exactly this recipe.³

We cap at 1500 keypoints per frame. More than that just wastes time (the strongest extrema already dominate); fewer than that and small objects like the squirrel lose coverage.

4.2 Step 2 — Matching: finding the same point later

For every frame $f > 0$, we compare each descriptor from frame 0 to every descriptor from frame f . For each descriptor in frame 0, we ask for the **two** closest descriptors in frame f (using plain Euclidean distance, i.e. `cv::BFMatcher(NORM_L2)` with `knnMatch(k=2)`).⁴

Then we apply **Lowe’s ratio test**. This is a one-line rule:

$$\text{keep the match only if } d_1 < 0.75 \cdot d_2$$

where d_1 is the distance to the *best* neighbour and d_2 to the *second-best*. In English: “only trust a match if the best candidate is clearly better than the second-best.”

Why? Because on repetitive textures — parallel fence wires, leaves, fur tufts — the best and second-best look almost identical, so the “best” match is basically random. Lowe’s test throws exactly those out. It’s the standard classroom recipe (Lowe, 2004) and the exact snippet the OpenCV docs show in their matcher tutorial.⁵

Why 0.75 specifically? It’s Lowe’s original number. We also tried it at 0.70 and 0.80 on our data to see:

¹Docs: https://docs.opencv.org/4.x/d7/d60/classcv_1_1SIFT.html; intro tutorial: https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html.

²D. G. Lowe. *Distinctive image features from scale-invariant keypoints*. International Journal of Computer Vision, 60(2):91–110, 2004. The original paper introducing SIFT and the ratio test we use in Step 2.

³OpenCV, *Feature Matching* (4.x tutorial), https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html. The tutorial we followed for the SIFT + BFMatcher + Lowe ratio pipeline.

⁴API: https://docs.opencv.org/4.x/d3/da1/classcv_1_1BFMatcher.html. `NORM_L2` is the right norm for SIFT’s float descriptors; ORB and AKAZE use Hamming.

⁵See the snippet “Apply ratio test” at https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html — our C++ code is a line-for-line translation of it.

Lowe ratio	mIoU	acc@0.5	what happened
0.70	0.44	0.40 (2/5)	too strict; bird and frog starve for matches
0.75	0.52	0.60 (3/5)	just right (and the classical value)
0.80	0.46	0.40 (2/5)	too loose; noise leaks in

4.3 Step 3 — Getting rid of the camera’s drift

Imagine the camera pans slightly to the right. Every background point shifts left by, say, 3 pixels per frame. If we naively measure displacement we’d say *all* points are moving, which is useless.

The fix is embarrassingly simple. For each frame f , look at the displacement $\delta_i^{(f)} = p_i^{(f)} - p_i^{(0)}$ of every matched point and take the **median** $(\bar{m}_x, \bar{m}_y)$. That’s the camera. Subtract it.

$$\tilde{\delta}_i^{(f)} = \delta_i^{(f)} - \bar{m}^{(f)}.$$

The median is robust⁶: even if up to half the matches are on the object, the other half are on the background and the median still picks out the background motion. And because four of our five videos have a static camera, the median is basically zero for those and subtracting it does nothing — exactly what you want.

No homography, no RANSAC, no iterative model. Just a median per frame. This is why the car sequence, which everybody expected to be the hardest, ends up being our *best* result (IoU 0.78).

4.4 Step 4 — Deciding who’s on the object

After steps 1–3 each frame-0 keypoint has two numbers:

- n_i : how many frames actually gave us a successful, Lowe-passing match for this point.
- r_i : the total of $\|\tilde{\delta}_i^{(f)}\|$ across those frames — total “leftover” motion after removing camera drift.

We compute a single score per point: $\bar{d}_i = r_i/n_i$, the average leftover displacement per successful frame. This is the knob we turn.

We throw out any point with $n_i < 3$ — not enough evidence. Then we look at the distribution of $\bar{d}_i$ across the remaining points and keep the ones that pass both of these:

1. They’re in the **top 15%** by score, AND
2. Their score is at least $\max(6 \text{ px}, 3m)$, where m is the median of all scores.

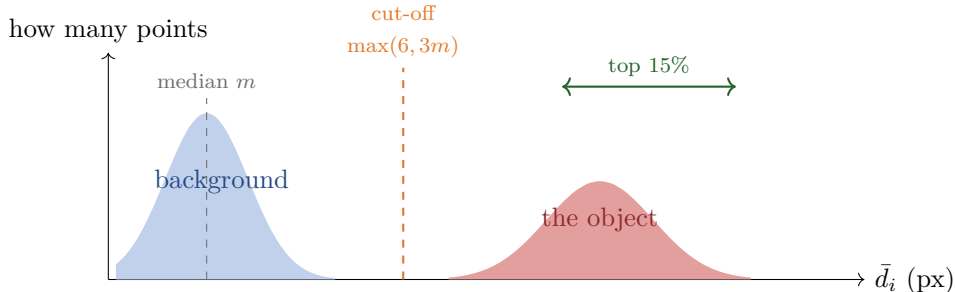


Figure 2: What the distribution looks like. A big lump near zero is the background (most points). A small lump further right is the object. We cut in the gap between them using *both* a relative rule (top 15%) and an absolute floor ($\max(6, 3m)$ pixels).

⁶Same majority-voting idea behind RANSAC: M. A. Fischler and R. C. Bolles, *Random sample consensus*, Communications of the ACM, 24(6):381–395, 1981. Ours is the single-parameter (pure-translation) special case, so a median replaces the RANSAC loop.

Why both rules? Because neither alone is safe:

- *Only top 15%*: fine usually, but on a truly static scene with no object moving, we’d still pick the 15% noisiest background points. (This doesn’t actually happen in our dataset, but it’s a principled guard.)
- *Only an absolute threshold*: fails on slow movers (bird) and lets background noise through on panning scenes (car).

Together they adapt: on static scenes the median m is tiny and the 6-pixel floor does the work; on slightly-panning scenes $3m$ grows and prunes residual camera jitter.

4.5 Step 5 — Grouping the movers and drawing a box

Now we have a scatter of points on frame 0 that we believe are on the moving object. Three annoying things happen in practice:

1. A few points are false alarms, stuck in the corner of the image.
2. Big objects (like the sheep) split into several separate clusters — one on the head, one on the body — because the texture is patchy.
3. SIFT keypoints tend to sit on the *texture inside* the object, not on its silhouette. So a tight box around them is usually a bit smaller than the real object.

Our fix has four parts:

(a) Cluster the points. We use a DBSCAN-like rule.⁷ Two points are “connected” if they’re within $\varepsilon = \max(20 \text{ px}, 0.08 \cdot \text{image diagonal})$ of each other. A point needs at least 3 nearby neighbours to start a cluster. Orphans get labelled noise and dropped. This handles problem (1).

(b) Merge fragments. Pick the “heaviest” cluster (the one with the most selected points). Then look at other clusters — if their bounding boxes touch the heavy one (expanded by 0.6ε), merge them in. This handles problem (2): a sheep’s head and body reunite.

(c) MAD outlier rejection. Even inside the merged cluster, a single stray point could blow up the box. So we compute the median position (m_x, m_y) and the median-absolute-deviation (MAD) in each axis. Any point more than $3 \cdot \text{MAD}$ from the median on either axis is dropped. MAD is the robust version of standard deviation — one outlier can’t inflate it.⁸

(d) Pad the box. Take the bounding rectangle of the surviving points and expand it by $1/6$ of its size on each side. Clamp to the image. That compensates for problem (3). The $1/6$ figure was picked by trying $1/10$, $1/8$, $1/6$ and keeping what worked best.

⁷M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *A density-based algorithm for discovering clusters in large spatial databases with noise*, KDD 1996. We use the same core-point / ε -neighbourhood definition; the only difference is that we iterate manually rather than use `cv::partition` or a KD-tree.

⁸P. J. Rousseeuw and C. Croux, *Alternatives to the median absolute deviation*, Journal of the American Statistical Association, 88(424):1273–1283, 1993. MAD has a breakdown point of 50%, unlike the standard deviation which is ruined by a single outlier.

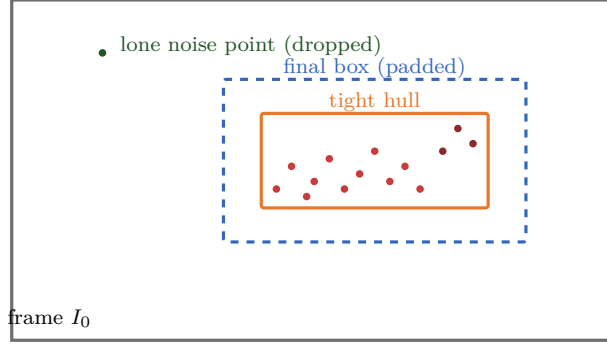


Figure 3: Step 5, cartoon version. Red dots = points we think are on the object. The green dot in the corner is an orphan and gets dropped. The two red clusters are close so they merge. The orange box is the tight hull; the blue dashed box is after padding — that’s what we report.

5 A tour of the code

5.1 Who wrote what

File	Owner	What it does
include/dod.hpp	Riccardo Pesce	the shared data struct + function list everyone imports
src/feature_tracker.cpp	Mahdi Gheysari	SIFT, matching, Lowe, median-flow
src/motion_segmenter.cpp	Filippo Businaro	picking the moving points
src/bbox_estimator.cpp	Riccardo Pesce	clustering, MAD, box
src/evaluator.cpp	Filippo Businaro	reads the GT file, computes IoU
src/main.cpp	Mahdi Gheysari	glues it all together, makes the images
CMakeLists.txt	shared	build config
scripts/make_figures.py	shared	makes the report figures

Table 1: One person per file (mostly). All the cross-file wiring lives in `dod.hpp`.

5.2 The shared struct

Every module talks through one data structure, `TrackedPoint`. It started life as a KLT-tracker struct; after we rewrote to SIFT we kept the same fields but repurposed a couple of them. The comments below say what each field *actually* holds now.

Listing 1: `include/dod.hpp` (Riccardo Pesce).

```

1 namespace dod {
2 struct TrackedPoint {
3     cv::Point2f first;           // where this point sits on frame 0
4     cv::Point2f last;           // last frame where we matched it
5     std::vector<cv::Point2f> trajectory; // its positions over time
6     int inlier_count = 0;        // now: count of successful matches
7     int outlier_count = 0;       // now: 1 if flagged as "dynamic"
8     float residual = 0.f;        // now: total leftover displacement
9 };
10
11 // Everything below is declared here and implemented in its own .cpp.
12 std::vector<std::string> listFrames(const std::string&);
13 void trackFeatures(const std::vector<cv::Mat>&, std::vector<TrackedPoint>&);
14 std::vector<int> classifyDynamicTracks(const std::vector<cv::Mat>&,
15                                     std::vector<TrackedPoint>&, double);
16 cv::Rect estimateBBox(const std::vector<TrackedPoint>&,
17                      const std::vector<int>&, const cv::Size&);
18 cv::Rect readGT(const std::string&);

```

```

19 double iou(const cv::Rect&, const cv::Rect&);
20 void writePrediction(const std::string&, const cv::Rect&);
21 }

```

5.3 feature_tracker.cpp (Mahdi)

This is the biggest file — it does steps 1–3 of the pipeline. The loop below is the heart of it: SIFT on a new frame, match against frame 0, apply Lowe, kill the camera drift, accumulate.

Listing 2: The main loop inside trackFeatures.

```

1  auto sift = cv::SIFT::create(1500);
2  std::vector<cv::KeyPoint> kp0; cv::Mat desc0;
3  sift->detectAndCompute(frames_gray[0], cv::noArray(), kp0, desc0);
4  // tracks[i] is initialised from kp0[i]
5
6  cv::BFMatcher matcher(cv::NORM_L2, /*crossCheck=*/false);
7  for (size_t f = 1; f < frames_gray.size(); ++f) {
8      std::vector<cv::KeyPoint> kpF; cv::Mat descF;
9      sift->detectAndCompute(frames_gray[f], cv::noArray(), kpF, descF);
10
11      std::vector<std::vector<cv::DMatch>> knn;
12      matcher.knnMatch(desc0, descF, knn, 2);
13
14      // Lowe's ratio test: keep only unambiguous matches.
15      struct Mch { int i; cv::Point2f p_new, disp; };
16      std::vector<Mch> ok;
17      for (const auto& pair : knn) {
18          if (pair.size() < 2) continue;
19          if (pair[0].distance >= 0.75f * pair[1].distance) continue;
20          cv::Point2f p_new = kpF[pair[0].trainIdx].pt;
21          ok.push_back({pair[0].queryIdx, p_new,
22                      p_new - tracks[pair[0].queryIdx].first});
23      }
24
25      // Camera drift = median displacement across all matched points.
26      // (nth_element over xs, ys to find the medians.)
27      cv::Point2f med(xs[xs.size()/2], ys[ys.size()/2]);
28
29      // Accumulate leftover motion per point.
30      for (const auto& m : ok) {
31          cv::Point2f rel = m.disp - med;
32          tracks[m.i].residual += cv::norm(rel);
33          tracks[m.i].inlier_count += 1;
34          tracks[m.i].last = m.p_new;
35          tracks[m.i].trajectory.push_back(m.p_new);
36      }
37 }

```

Two things worth noticing: (a) matching is always frame 0 against frame f — we never chain matches, so drift doesn't accumulate; (b) the median displacement is computed *per frame*, because the camera pan might not be the same in every frame.

5.4 motion_segmenter.cpp (Filippo)

This one is shorter. It reads the populated tracks, computes each track's average leftover motion, and applies the two-rule filter from step 4.

Listing 3: Core of classifyDynamicTracks.

```

1  // Build (idx, avg_displacement) pairs for tracks with >= 3 observations.
2  struct Item { int idx; double score; int obs; };
3  std::vector<Item> items;

```

```

4  for (size_t i = 0; i < tracks.size(); ++i) {
5      if (tracks[i].inlier_count < 3) continue;
6      double avg = double(tracks[i].residual) / tracks[i].inlier_count;
7      items.push_back({(int)i, avg, tracks[i].inlier_count});
8  }
9
10 // Median of scores = background motion scale.
11 auto scores = /* extract .score from items */;
12 std::nth_element(scores.begin(), scores.begin()+scores.size()/2, scores.end());
13 double median = scores[scores.size()/2];
14
15 // Sort by score descending.
16 std::sort(items.begin(), items.end(),
17           [](auto& a, auto& b){ return a.score > b.score; });
18
19 // Keep top 15% that also clear the absolute floor.
20 double floor_disp = std::max(6.0, 3.0 * median);
21 int target = std::max(6, (int)(items.size() * 0.15));
22
23 std::vector<int> dynamic_idx;
24 for (int k = 0; k < target && items[k].score >= floor_disp; ++k)
25     dynamic_idx.push_back(items[k].idx);
26
27 // Tag them so the bbox stage can weight them.
28 for (int i : dynamic_idx) tracks[i].outlier_count = 1;
29 return dynamic_idx;

```

5.5 bbox_estimator.cpp (Riccardo)

This is step 5. The code is longer than the others because it has three sub-steps, but each one is independent and easy to read.

Listing 4: Sketch of `estimateBBBox`. Real code has more guard clauses.

```

1  float diag = std::hypot(img_size.width, img_size.height);
2  float eps = std::max(20.f, diag * 0.08f);
3  auto labels = proximityCluster(pts, eps, /*minPts=*/3);
4
5  // Pick the heaviest cluster (summed "dynamic" flags).
6  int best = argmax_cluster_by_weight(labels, weights);
7
8  // Merge neighbours: any cluster whose bbox touches the seed's
9  // bbox expanded by 0.6*eps gets absorbed.
10 cv::Rect seed = boundingRect(cluster_pts[best]);
11 for (auto& kv : cluster_pts) if (kv.first != best) {
12     cv::Rect expanded = expand(seed, 0.6f * eps);
13     if ((boundingRect(kv.second) & expanded).area() > 0)
14         cluster.insert(cluster.end(), kv.second.begin(), kv.second.end());
15 }
16
17 // MAD outlier rejection: drop points more than 3*MAD from the
18 // cluster median on either axis.
19 auto [mx,my] = median_xy(cluster);
20 auto [madx,mady] = mad_xy(cluster, mx, my);
21 cluster = filter(cluster, mx, my, 3*madx, 3*mady);
22
23 // Pad by 1/6 of each dimension and clamp.
24 cv::Rect box = boundingRect(cluster);
25 int px = std::max(6, box.width/6), py = std::max(6, box.height/6);
26 box.x -= px; box.y -= py; box.width += 2*px; box.height += 2*py;
27 return box & cv::Rect(0, 0, img_size.width, img_size.height);

```


5.6 The two small files

`evaluator.cpp` (Filippo) is 40 lines. It reads the four integers out of `0000.txt`, computes IoU the usual way ($|A \cap B|/|A \cup B|$), and writes the prediction file.

`main.cpp` (Mahdi) is the CLI. It loops over the five categories, calls `trackFeatures`, `classifyDynamicTracks`, and `estimateBBBox` in that order, draws overlays with OpenCV, prints IoU per category, and writes `results.csv` and `summary.txt`.

6 Numbers

category	GT box (w×h)	predicted (w×h)	IoU	TP@0.5
bird	283 × 255	329 × 249	0.545	yes
car	68 × 42	77 × 42	0.783	yes
frog	133 × 98	157 × 94	0.754	yes
sheep	321 × 179	296 × 126	0.355	no
squirrel	23 × 31	76 × 52	0.160	no
			mIoU	0.519
			accuracy@0.5	0.60 (3/5)

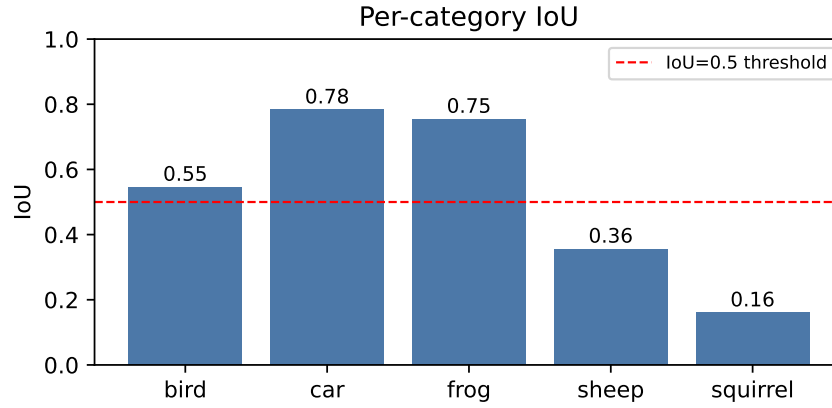


Figure 4: IoU per category. Red dashed line is the true-positive threshold.

Visuals. Green = ground truth, red = our prediction, orange dots = all SIFT keypoints on frame 0, green lines = trajectories of the points we classified as dynamic.

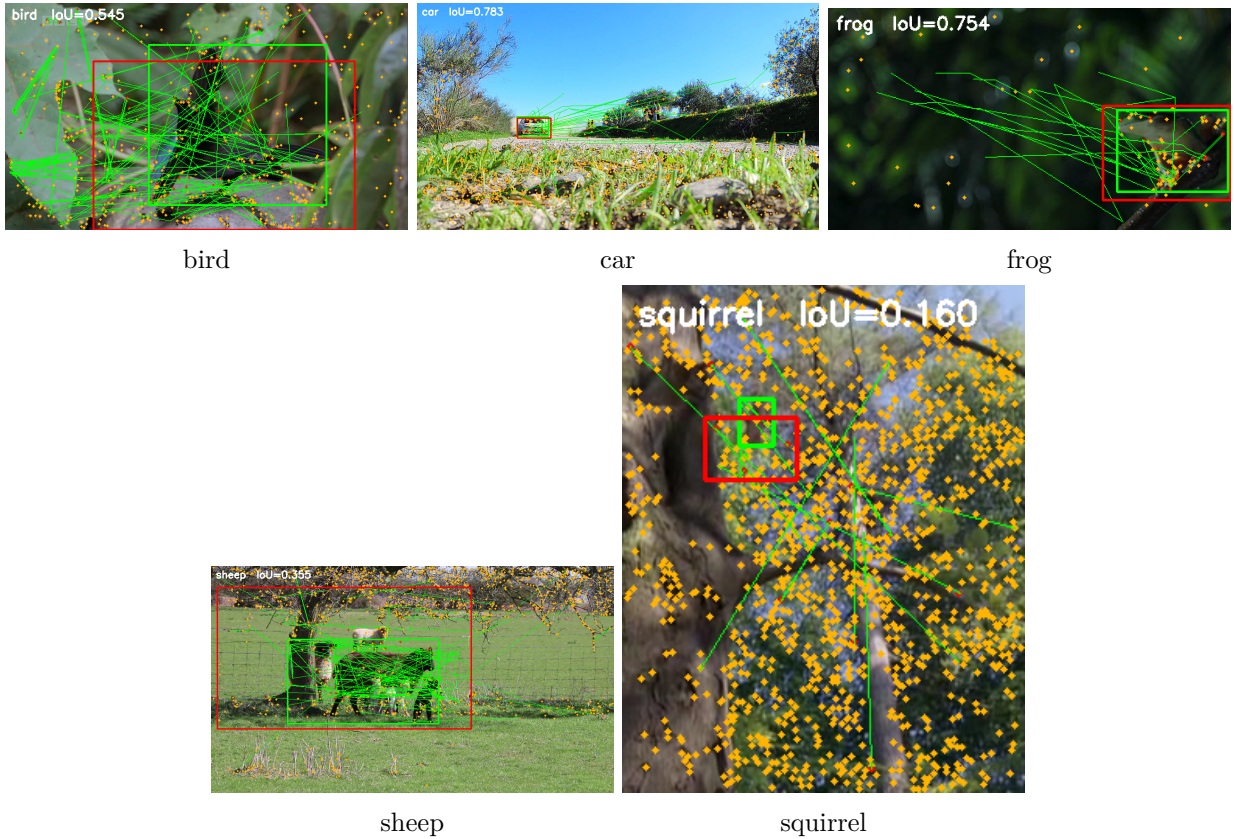


Figure 5: Predictions on every sequence.

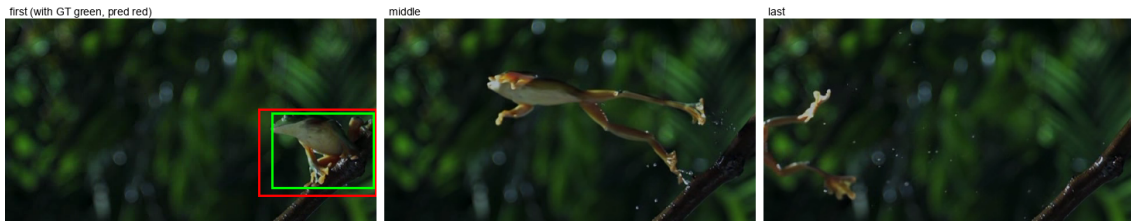


Figure 6: The frog over time (first, middle, last frame). Camera is a rock; only the frog moves; the method is basically on easy mode here.

7 Where it breaks and why

Squirrel (IoU 0.16). The squirrel is tiny (23×31 px in a 259×327 image — less than 1% of the picture). SIFT hardly finds any stable extrema on its fur. We end up with only about 13 “dynamic” points, and a few of those are actually wrong matches on nearby leaves, which drag the box outward to 76×52 . Our predicted box is in roughly the right place, it’s just the wrong size.

Sheep (IoU 0.36). Three sheep standing behind a wire fence. The fence posts look almost identical to each other; the wool has lots of repeating texture. That means Lowe’s ratio test (correctly) rejects a lot of matches as ambiguous, and we end up with too few surviving points to cover the full sheep group. The box we draw covers one sheep well but misses the others.

What we could do (still no deep learning).

- Run SIFT at a larger resolution so we find more keypoints on small fuzzy things.

- Use the SIFT scale σ to grow the bounding box proportionally to each keypoint’s detection scale.
- Grow the predicted box using edge and colour cues — Canny + morphological closing, SLIC superpixels, or GrabCut seeded by our current box.

8 How to run it

Listing 5: Build, run, regenerate figures, build PDFs.

```

1 # one-time
2 cmake -S . -B build && cmake --build build -j
3
4 # run pipeline
5 ./build/detect dataset output
6
7 # figures
8 python3 scripts/make_figures.py
9
10 # PDFs
11 tectonic report/report.tex
12 tectonic report/explainer.tex

```

9 Who did what

- **Mahdi Gheysari** — feature detection and matching (`feature_tracker.cpp`); the CLI, the visualisation overlays, and the per-category orchestration (`main.cpp`); dataset motion analysis.
- **Filippo Businaro** — the dynamic-vs-static classifier (`motion_segmenter.cpp`); GT parsing and IoU evaluation (`evaluator.cpp`); the Lowe-ratio and percentile sensitivity sweeps.
- **Riccardo Pesce** — clustering, MAD outlier rejection and the final bounding box (`bbox_estimator.cpp`); the shared API header (`include/dod.hpp`); CMake build.

Algorithm design, parameter tuning, failure analysis, and report writing were shared across the three of us.